PREMIER UNIVERSITY CHATTOGRAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Complex Engineering Problem

COURSE NAME	Cloud Computing
COURSE CODE	CSE 4523
ASSIGNMENT NAME	**A Cloud–Edge–IoT Traffic Management System with AI Optimization**
DATE OF ASSIGNMENT	11-04-2026
DATE OF SUBMISSION	-04-2026

SUBMITTED TO

Kingshuk Dhar
Assistant Professor & Administrative Coordinator
Department Of Computer Science and Engineering

REMARKS	**SUBMITTED BY**	
	NAME	Rimjhim Dey
	ID	0222220005101039
	SEMESTER	7th
	BATCH	42nd
	SESSION	Fall 2025
	SECTION	A

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Background and Motivation 3
2.2 Scope and Objectives 4

3 System Architecture Overview **4**
3.1 Three-Tier Architecture 4

4 IoT Layer — Sensor Network Design **6**
4.1 Device Categories 6
4.2 Data Volume Estimation 6

5 Edge Computing Layer **7**
5.1 Edge Node Architecture 7
5.2 Edge Processing Pipeline 7
5.3 Modified Webster's Adaptive Formula 8

6 Cloud Infrastructure — AWS Design **8**
6.1 AWS Service Selection 8
6.2 Data Pipeline Architecture 9
6.3 Deployment Model: Hybrid Cloud 10
6.4 Service Model 10

7 AI and Machine Learning Models **10**
7.1 Traffic Congestion Prediction — LSTM 10
7.2 Adaptive Signal Control — Reinforcement Learning 10
7.3 Incident Detection — Computer Vision 11
7.4 Route Optimisation — Dijkstra + Heuristic 11

8 Data Flow Diagrams and Flowcharts **11**
8.1 End-to-End Data Flow 11
8.2 Emergency Response Flowchart 13
8.3 AI Model Training and Deployment Lifecycle 15

9 Security and Compliance **16**
9.1 Security Architecture 16
9.2 Communication Protocols 17

10 Scalability and Fault Tolerance **17**
10.1 Scalability Strategy 17
10.2 Fault Tolerance Mechanisms 18

11 AWS Cost Estimation **19**
11.1 Assumptions 19
11.2 Monthly Cost Breakdown 20
11.3 Cost Optimisation Strategies 20
11.4 Cost Visualisation 21

12 Evaluation and Design Justification **22**
12.1 Cloud Infrastructure Choices 22
12.2 Addressing Conflicting Requirements 23
12.3 Standards Compliance 23

13 Infrequent Challenges and Mitigation **24**
13.1 Sensor Failures and Calibration Drift 24
13.2 Network Instability in Semi-Urban Areas 24
13.3 Large-Scale Real-Time Streaming Data 24

14 Implementation Roadmap **25**

15 Conclusion **25**

1 Abstract

This assignment presents a comprehensive design of an **Intelligent Smart Transportation and Traffic Management System (ISTMS)** tailored for metropolitan and semi-urban areas in Bangladesh. The system addresses severe urban congestion challenges in cities such as Dhaka, Chittagong, Sylhet, and Rajshahi, where traffic congestion causes annual economic losses exceeding BDT 37,000 crore (approximately USD 4.4 billion).

The proposed system integrates a three-tier architecture comprising: (1) IoT-enabled sensor networks (CCTV cameras, inductive loop sensors, GPS trackers, and smart traffic signals) that collect real-time traffic data; (2) edge computing nodes deployed at roadside units (RSUs) that perform low-latency AI inference for signal control and incident detection; and (3) a scalable AWS cloud backend for large-scale analytics, model training, and long-term storage.

Key components include smart adaptive traffic signals, CCTV-based incident detection using YOLOv8, GPS-tracked public transport for real-time arrival predictions, and AI-driven congestion prediction models using Long Short-Term Memory (LSTM) networks with Deep Q-Network (DQN) reinforcement learning for adaptive signal optimisation. A cloud-edge hybrid deployment model ensures sub-100 ms response times for real-time signal adjustments while leveraging AWS services (EC2, S3, DynamoDB, Kinesis, Lambda, and SageMaker) for centralised model training and long-term planning.

Monthly operational costs are estimated at approximately **USD 3,352** (plus 20% contingency = **USD 4,022**) using the AWS Pricing Calculator, with a roadmap for cost reduction through spot instances, reserved instances, and edge pre-processing that saves over $8,000 per month in avoided data transfer costs.

2 Introduction

2.1 Background and Motivation

Bangladesh, with a population exceeding 170 million and a rapidly urbanising economy, faces some of the most acute urban traffic challenges in South Asia. Dhaka, the capital city, consistently ranks among the world's most congested cities, with average traffic speeds dropping below 7 kilometres per hour during peak periods. According to a World Bank assessment published in 2022, Dhaka alone loses an estimated **3.2 million productive working hours daily** to traffic congestion, equivalent to an annual economic loss of approximately BDT 37,000 crore (USD 4.4 billion).

The existing traffic management infrastructure in Bangladeshi cities is fundamentally inadequate. Current systems rely predominantly on static, fixed-cycle signal controllers that operate on predetermined timing plans regardless of actual traffic conditions. Traffic management also depends heavily on manual police intervention at intersections, an approach that is neither scalable nor consistently effective.

Current limitations include:

- **Lack of real-time visibility:** No citywide sensor network exists to quantify vehicle density, monitor speeds, or detect incidents automatically. Traffic management decisions are made based on limited observation rather than comprehensive data.
- **Static signal plans:** Fixed green/red cycles ignore actual traffic demand, resulting in situations where vehicles wait at red lights while no cross traffic exists, or where heavily congested approaches receive insufficient green time.
- **Delayed emergency response:** Ambulances carrying critical patients, fire trucks responding to fires, and police vehicles lose precious minutes navigating gridlocked intersections without priority routing.
- **Absence of data-driven planning:** Long-term infrastructure decisions rely on periodic manual traffic surveys rather than continuous, citywide data streams, leading to suboptimal investment decisions.

2.2 Scope and Objectives

The ISTMS addresses these gaps through five primary objectives:

1. **Real-time traffic monitoring** via IoT sensors and cameras to provide complete situational awareness.
2. **AI-based traffic prediction and optimisation** with adaptive signal control to reduce delays and improve throughput.
3. **Scalability and fault tolerance** across multi-city deployment from pilot to nationwide coverage.
4. **Public safety and emergency handling** through automated incident detection and signal pre-emption.
5. **Cost efficiency** via optimised edge-cloud workload distribution that balances performance with operational expenses.

3 System Architecture Overview

3.1 Three-Tier Architecture

The ISTMS adopts a **Cloud-Edge-IoT three-tier architecture** that strategically balances four key design considerations: latency requirements, bandwidth constraints, computational capacity, and operational cost. Figure 1 illustrates the overall design showing the three distinct layers and the interactions between them.

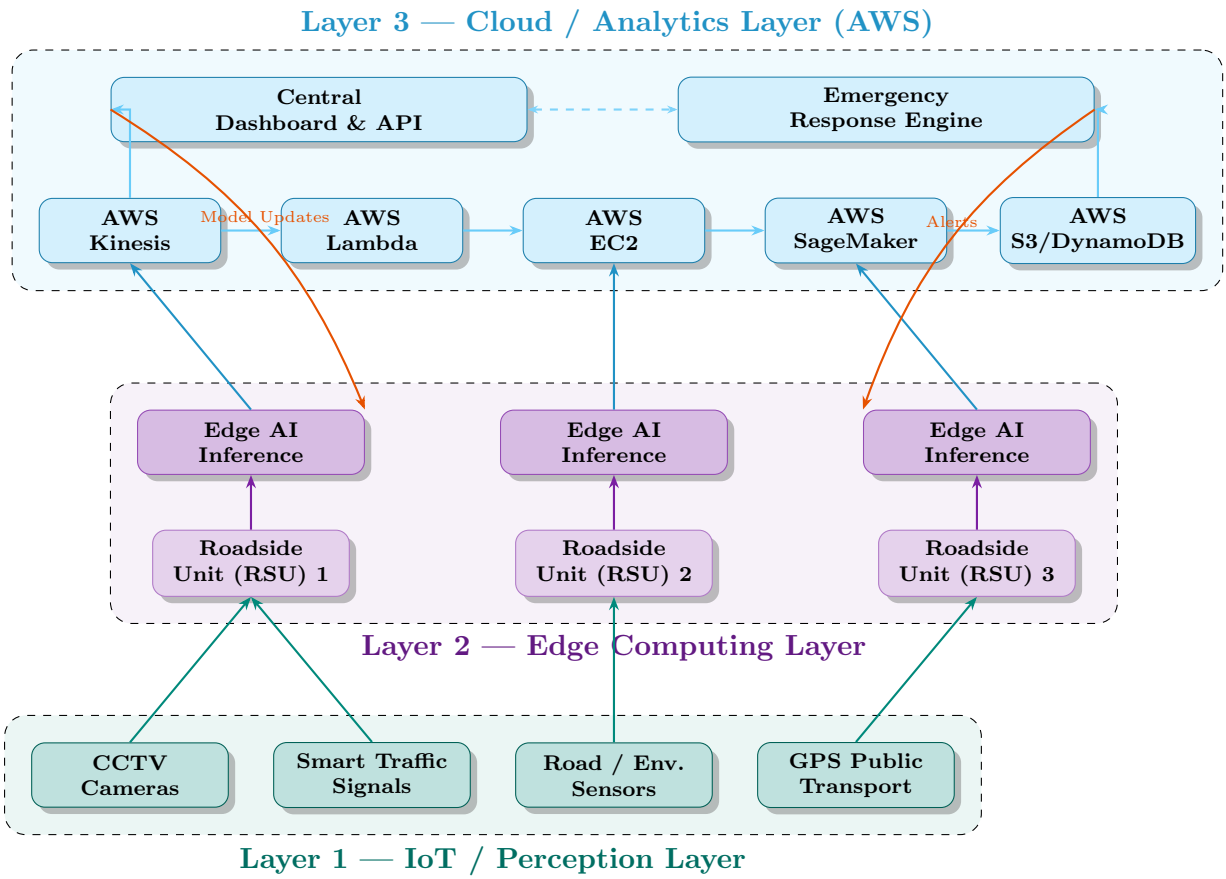

Figure 1: Three-Tier Architecture of the ISTMS showing IoT perception layer (bottom), edge computing layer (middle), and AWS cloud analytics layer (top).

The three layers serve distinct and complementary roles:

- **Layer 1 – IoT/Perception Layer:** Physical sensing devices (cameras, sensors, GPS units, smart signals) deployed at intersections and along road corridors. Data is transmitted over **MQTT/LTE-V2X** to the nearest edge node. This layer handles data collection and initial pre-processing.

- **Layer 2 – Edge Computing Layer:** Roadside Units (RSUs) host lightweight AI inference models for immediate (¡100 ms) traffic signal adjustments and incident detection. Each RSU runs containerised microservices on NVIDIA Jetson-class hardware, performing YOLOv8 object detection, LSTM congestion prediction, and adaptive signal control.

- **Layer 3 – Cloud/Analytics Layer (AWS):** Centralised data pipeline built on Kinesis, Lambda, EC2, SageMaker, S3, and DynamoDB for model training, long-term analytics, and the citywide dashboard. This layer also hosts the emergency response coordination engine.

4 IoT Layer — Sensor Network Design

4.1 Device Categories

Table 1 provides a comprehensive inventory of IoT devices deployed in the system, including the type of data each captures, the communication protocol used, and the typical deployment location.

Table 1: IoT Device Inventory for ISTMS

Device Type	Data Captured	Protocol	Deployment Location
CCTV / Smart Camera	Video (1080p), vehicle count, speed	RTSP over LTE	Major intersections, highways
Inductive Loop Sensor	Vehicle count, speed, headway	MQTT over LTE	Embedded in asphalt at stop lines
Smart Traffic Signal	Phase state, queue length, call buttons	MQTT/CoAP	All signalised intersections
Environmental Sensor	PM2.5, temperature, humidity, rain	MQTT	Roadside poles, signal mast arms
GPS Tracker (Bus/CNG)	Location, speed, occupancy status	MQTT over 4G	Public transport fleet vehicles
Ultrasonic Sensor	Pedestrian presence, crossing events	CoAP	Pedestrian crossings, near schools
RFID / Number Plate Reader	Vehicle identity, travel time	HTTP REST	Toll plazas, city entry points
Emergency Beacon	Ambulance/fire truck location, status	MQTT over LTE	All emergency response vehicles

4.2 Data Volume Estimation

Understanding the data volume that the system must handle is essential for designing network infrastructure, edge storage, and cloud data pipelines. The following estimates are based on a full Dhaka deployment with 500 signalised intersections:

- **Cameras:** 500 intersections $\times$ 2 cameras $\times$ 512 Kbps (compressed H.265) = 512 Mbps aggregate bandwidth. Video is processed locally at the edge and never transmitted to the cloud.

- **Sensors:** 2,000 devices × 1 reading/second × 200 bytes ≈ 34.6 GB/day of sensor data.
- **GPS Trackers:** 10,000 vehicles × 1 update/5 seconds × 100 bytes ≈ 17.3 GB/day of location data.
- **Total estimated ingestion to cloud:** Approximately 52 GB/day or **1.6 TB/month** of metadata. Raw video (which would be terabytes per day) stays at the edge, representing a massive cost saving.

5 Edge Computing Layer

5.1 Edge Node Architecture

Each Roadside Unit (RSU) is designed as a complete computing system capable of running the real-time processing pipeline independently of cloud connectivity. Figure 2 shows the software stack within a single RSU, organised in layers from hardware at the bottom to cloud synchronisation at the top.

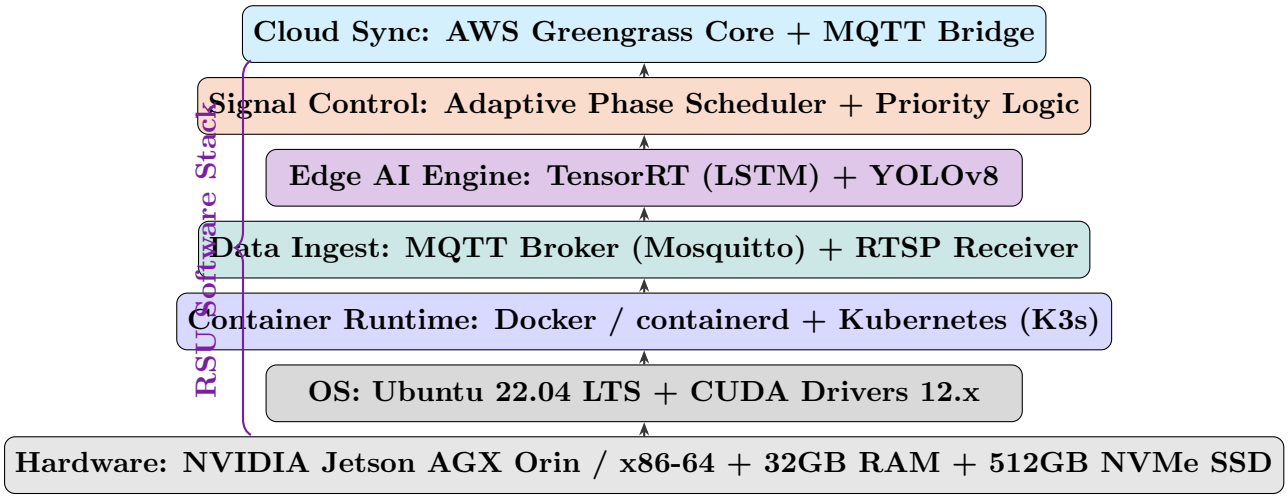

Figure 2: Software stack of a single Roadside Unit (RSU) in the Edge Computing Layer.

5.2 Edge Processing Pipeline

The edge processing pipeline executes a sequence of operations on each incoming data frame with strict latency requirements. The complete pipeline from sensor input to signal command must complete within 100 milliseconds:

1. Raw sensor data and video frames arrive at the MQTT broker and RTSP receiver respectively.
2. The **YOLOv8-nano** computer vision model processes each camera frame to detect vehicles, count density, and flag incidents (accidents, wrong-way drivers, pedestrian hazards) within <**50 ms**.

3. The **LSTM inference engine** (quantised INT8 via TensorRT) predicts congestion for the next 5-minute window using the last 30 minutes of sensor readings, completing inference in approximately 15 ms.

4. The **Adaptive Phase Scheduler** combines current queue lengths with LSTM predictions to compute optimal green-time allocations using a Modified Webster's formula.

5. Signal commands are dispatched to smart traffic signals via low-latency MQTT publish with QoS Level 2 (exactly-once delivery).

6. Aggregated metadata (vehicle counts per minute, average speeds, incident flags) is forwarded to the AWS cloud via Greengrass. Raw video never leaves the edge.

5.3 Modified Webster's Adaptive Formula

The optimal cycle length C^* at each RSU is computed using a modified Webster's formula that incorporates LSTM predictions:

$$C^* = \frac{1.5L + 5}{1 - \sum_{i=1}^{n} Y_i} \cdot \alpha(\hat{\rho}) \tag{1}$$

where:

- L is the total lost time per cycle (seconds) due to phase transitions
- $Y_i = q_i/s_i$ is the flow ratio for phase i (demand divided by capacity)
- $\alpha(\hat{\rho})$ is an LSTM-derived scaling factor based on predicted density $\hat{\rho}$

When predicted density $\hat{\rho}$ is high (indicating impending congestion), α is set >1.0 to extend cycle length. When density is low, $\alpha < 1.0$ allows shorter cycles that reduce wait times.

6 Cloud Infrastructure — AWS Design

6.1 AWS Service Selection

Figure 3 maps each functional requirement to the selected AWS service, leveraging managed services to reduce operational overhead.

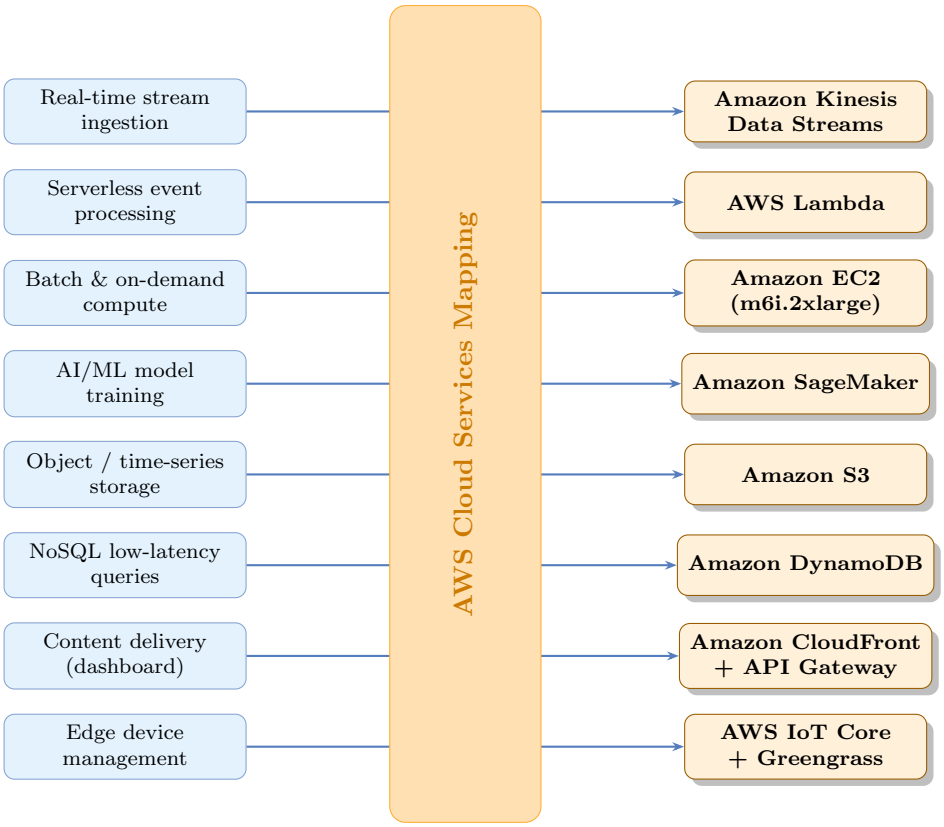

Figure 3: Mapping of ISTMS functional requirements to AWS cloud services.

6.2 Data Pipeline Architecture

The end-to-end cloud data pipeline processes data from the moment it leaves edge nodes to final storage and consumption:

1. **AWS IoT Core** receives MQTT messages from Greengrass agents on each RSU (≈200,000 messages/min citywide), authenticating each device using X.509 certificates.

2. **Kinesis Data Streams** buffers real-time telemetry into scalable shards, enabling fan-out to multiple downstream consumers.

3. **AWS Lambda** (event-driven) performs anomaly detection using Z-score analysis on speed data and triggers SNS alerts for incidents.

4. **EC2 Auto Scaling Group** runs Apache Flink for 1-minute micro-batch aggregations, writing results to DynamoDB (hot path) and S3 (cold path).

5. **SageMaker Pipelines** re-trains LSTM and DQN models nightly on the S3 data lake; updated model artifacts are pushed to Greengrass for edge deployment.

6. **CloudFront + API Gateway** serve the React-based central dashboard and REST APIs consumed by traffic police and government agencies.

6.3 Deployment Model: Hybrid Cloud

The ISTMS employs a **Hybrid Cloud** deployment model, distributing resources between public cloud and on-premise edge:

- **Public Cloud (AWS ap-southeast-1, Singapore):** Large-scale analytics, model training, dashboard, and long-term storage.
- **On-Premise Edge (RSUs at intersections):** Real-time signal control, incident response, and privacy-sensitive video processing (video never leaves the edge).
- **Rationale:** Video data contains PII; processing locally ensures compliance with Bangladesh's Digital Security Act 2018. Low-latency signal control (<100 ms) is physically impossible with cloud round-trip latency (∼120 ms to Singapore).

6.4 Service Model

The system utilises multiple cloud service models:

- **IaaS**: EC2 instances running Flink, API servers, and edge RSU bare-metal provisioning.
- **PaaS**: SageMaker (managed ML pipeline), Kinesis (managed streaming), DynamoDB (managed NoSQL), Lambda (serverless functions).
- **SaaS**: Central dashboard delivered as a web application — "Traffic-as-a-Service."

7 AI and Machine Learning Models

7.1 Traffic Congestion Prediction — LSTM

A many-to-one **Long Short-Term Memory (LSTM)** network predicts traffic density index $\hat{\rho}(t + \Delta)$ for the next 5-minute window:

$$\hat{\rho}(t + \Delta) = f_{\text{LSTM}}(\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t;\ \theta) \tag{2}$$

where $\mathbf{x}_t = [\rho_t,\ v_t,\ q_t,\ \text{time}_t,\ \text{weather}_t]^\top$ is the input feature vector, $T = 30$ (30-minute lookback), and θ are trainable parameters optimised by AdamW with MSE loss.

Model architecture: 2 stacked LSTM layers (128, 64 hidden units) + Dropout(0.2) + Dense(1). Training uses 18 months of historical sensor data from Dhaka's 200 major intersections. Target accuracy: MAE $< 8\%$ on density index (scale 0–1).

7.2 Adaptive Signal Control — Reinforcement Learning

A **Deep Q-Network (DQN)** agent optimises signal phase allocation:

- **State** s_t: queue lengths on all approach lanes, current phase, time-of-day.
- **Action** a_t: extend current green phase by 5 seconds or switch to next phase.
- **Reward** $r_t = -\sum_i w_i L_i(t)$, where L_i is queue length on lane i.
- Trained in SUMO (Simulation of Urban MObility) environment with real Dhaka road geometry.

7.3 Incident Detection — Computer Vision

YOLOv8-nano (edge-quantised to INT8) processes camera frames:

- Detects: accidents, stationary vehicles, pedestrian incursions, wrong-way driving.
- Inference time: ≈25 ms per frame on Jetson Orin at 30 FPS.
- Triggers SNS/Lambda alert pipeline within <3 seconds of incident onset.

7.4 Route Optimisation — Dijkstra + Heuristic

For emergency vehicles, a **modified Dijkstra algorithm** computes shortest-time path on live road graph $G = (V, E)$ where edge weights $w_{ij}(t)$ update every 30 seconds from real-time density data. Signal pre-emption activates 60 seconds ahead of emergency vehicle along its computed route.

8 Data Flow Diagrams and Flowcharts

This section presents three key diagrams describing the operational behaviour of the ISTMS: the end-to-end data flow across all three system layers, the automated emergency response decision logic, and the MLOps lifecycle for continuous model improvement. Each diagram is accompanied by a walkthrough explaining the key events and decisions at each stage.

8.1 End-to-End Data Flow

Figure 4 traces the journey of traffic data from physical sensors through to a model update pushed back to the edge. The flow starts at the **IoT Sensors** node, where cameras, inductive loops, GPS units, and environmental sensors capture raw data. Each device applies minimal on-device filtering before transmitting over MQTT or LTE to its Roadside Unit. Inside the **Edge Layer** (shown in purple), the RSU's first action is to evaluate the diamond-shaped "Incident Detected?" decision node. If YOLOv8 flags an incident above its confidence threshold, the flow branches right and an **SNS emergency alert** is published in parallel with the normal pipeline. Otherwise, the RSU runs the **Edge AI Inference** block — executing the LSTM congestion forecast and confirming

vehicle counts — then immediately feeds the results into the **Adaptive Signal Phase Update**. The Modified Webster's formula uses the LSTM output and current queue lengths to compute green-time allocations and dispatch signal commands. This entire edge loop must complete within 100 ms, which is precisely why it cannot be offloaded to the cloud.

Once the local decision is made, the RSU forwards a compact metadata package (counts, speeds, incident flags) up to the **Cloud Layer** (shown in blue). **AWS Kinesis** buffers the stream and fans it out to downstream consumers: **Lambda** performs a city-wide Z-score anomaly check, while the **EC2/Flink** cluster runs one-minute micro-batch aggregations. Results are written to **DynamoDB** (hot store for live dashboard queries) and **S3** (cold store for historical analysis). Nightly, **SageMaker** re-trains the LSTM and DQN models on the accumulated S3 data and pushes updated model artefacts back to all RSUs via Greengrass OTA — completing the feedback loop represented by the dashed orange arrow.

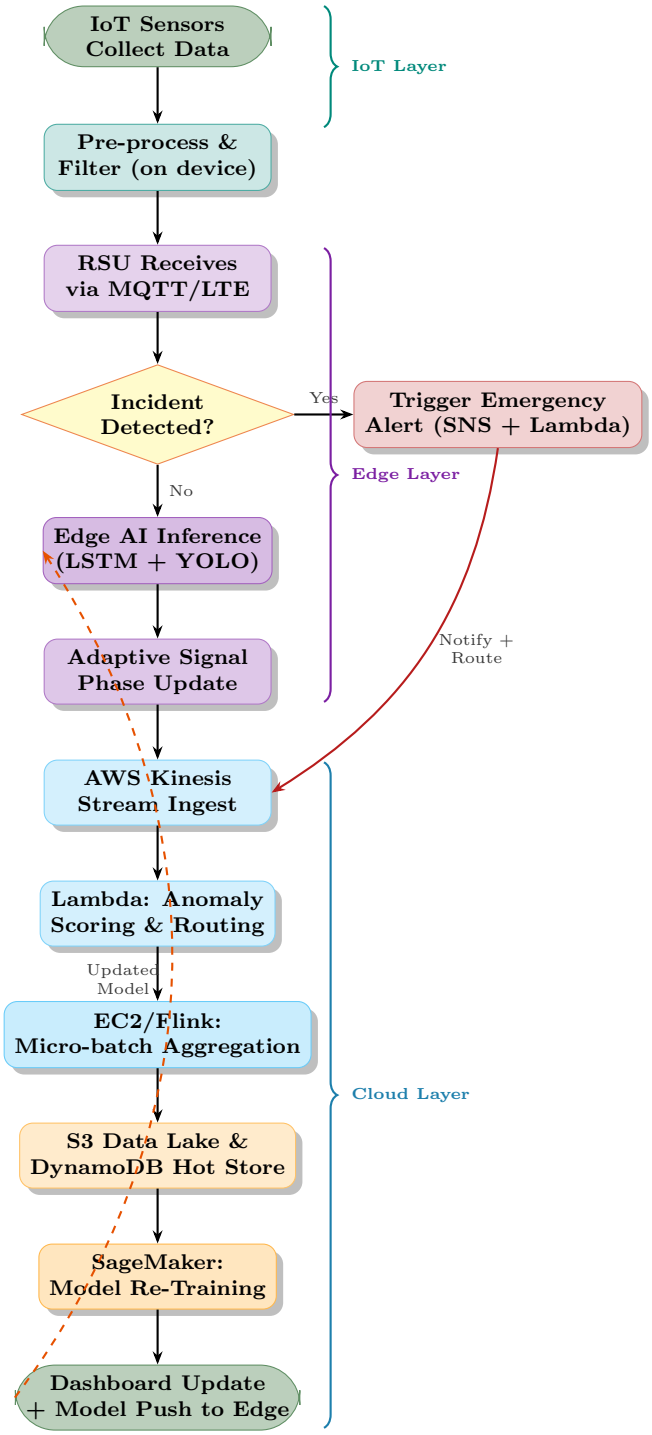

Figure 4: End-to-end data flow diagram from IoT data collection through edge processing to cloud analytics. The dashed orange arrow represents the model update feedback loop from SageMaker back to the edge RSUs.

8.2 Emergency Response Flowchart

Figure 5 shows the automated decision process triggered whenever YOLOv8 flags a potential road emergency. The flow begins with the "Confidence > 85%?" decision node. A single-frame detection is insufficient to raise an alert; the system waits for three consecutive frames to confirm the incident above threshold, which prevents false positives

from shadows, reflections, or fast-moving objects. Once confirmed, the RSU publishes an **SNS alert** carrying the incident type and GPS coordinates, which a Lambda function immediately receives and uses to classify severity into one of three levels. **Level 1** (minor obstruction) triggers only a signal phase advisory to surrounding intersections. **Level 2** (significant blockage or wrong-way driver) dispatches the nearest traffic police unit via the integrated dispatch API. **Level 3** (confirmed injury accident or vehicle fire) initiates a 999 call to the National Emergency Communications Centre with full incident data.

All three severity branches then converge at the **Compute Optimal Route** node, which runs the modified Dijkstra algorithm on the live road graph — weighting edges with the most recent density readings — to find the fastest path from the nearest depot to the incident site. The computed route feeds into **Signal Pre-emption**, which pushes green-phase commands to every signalised intersection along the corridor so that a wave of green lights is ready as the emergency vehicle approaches. Once the incident is cleared by a human operator or confirmed resolved by the camera feed, the system logs all response times and closes the ticket.

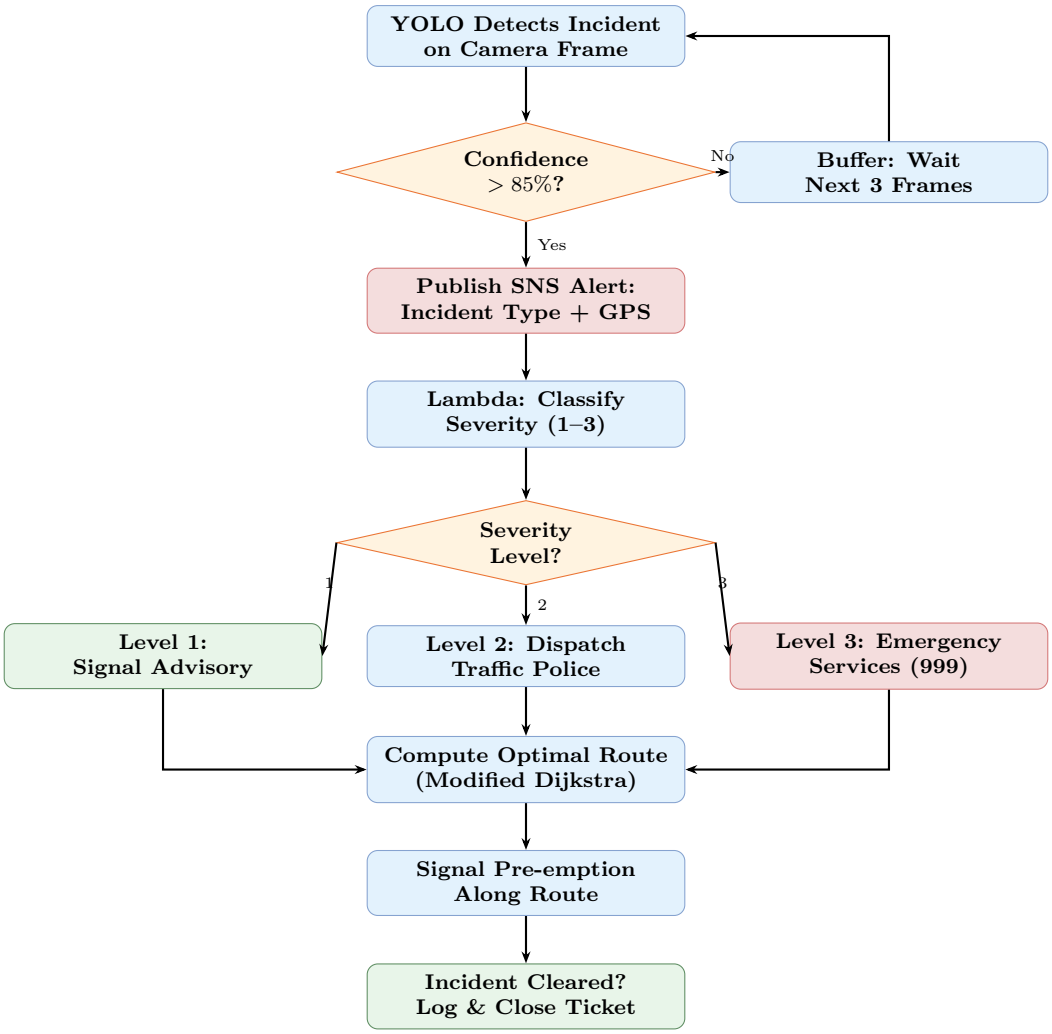

Figure 5: Automated emergency response flowchart. The process begins with YOLOv8 incident detection, applies a multi-frame confidence check to avoid false positives, classifies severity into three levels, and activates signal pre-emption along the optimal emergency route.

8.3 AI Model Training and Deployment Lifecycle

Figure 6 shows how the ISTMS's AI models are kept continuously up to date. A model trained once at deployment would degrade quickly as traffic patterns shift with new roads, seasonal demand, and population growth, so the pipeline automates the full cycle from data ingestion to edge deployment. The cycle begins with **Data Collection** in the S3 data lake, followed by a **Data Validation** step that applies schema checks and distribution tests to catch corrupted readings from sensor failures or network dropouts before they enter training. Validated data passes through **Feature Engineering** — converting raw counts, speeds, and timestamps into LSTM-ready feature vectors (rolling averages, cyclical time encodings, weather flags) — before the **Model Training** job runs on a SageMaker GPU instance (ml.p3.2xlarge) for approximately 30 minutes using the previous seven days of rolling data.

The newly trained model is tested in **Model Evaluation** against a held-out validation set; any model that does not meet the MAE $< 8\%$ threshold for LSTM or the 2% reward-improvement threshold for DQN is discarded and the previous production model is retained. Models that pass are versioned in the **Model Registry** and deployed to all RSUs via **Greengrass OTA**, which hot-swaps the inference engine without interrupting live signal control. Post-deployment, **CloudWatch** monitors prediction accuracy and inference latency in real time. When the **Drift Detection** module detects that live prediction error has diverged significantly from the validation baseline — signalling that real-world traffic patterns have shifted — it automatically fires a retraining trigger, restarting the cycle.

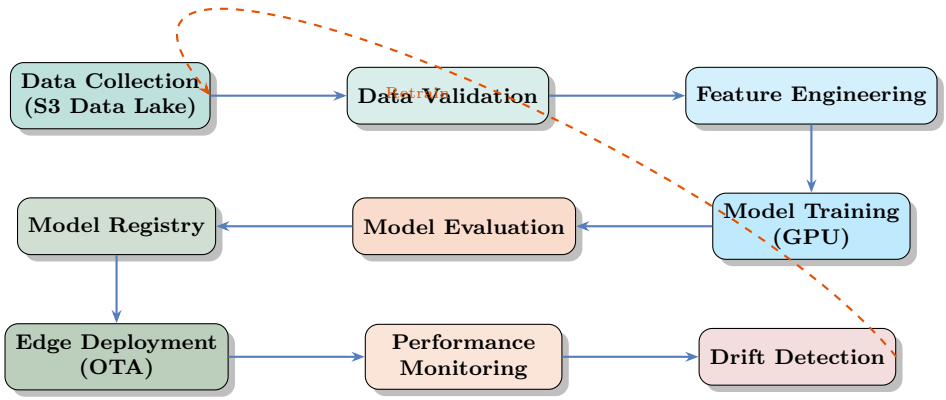

Figure 6: MLOps pipeline for continuous LSTM and DQN model training, evaluation, and edge deployment. The dashed arrow represents the automatic retraining trigger activated by the drift detection module when model performance degrades below threshold.

9 Security and Compliance

9.1 Security Architecture

The ISTMS enforces a multi-layer security model that protects the system at every boundary — from the physical IoT device through to the cloud — so that a compromise at any single point does not cascade into a system-wide failure.

At the **device level**, each IoT sensor and RSU is provisioned with a unique X.509 digital certificate issued by the AWS IoT Core Certificate Authority. RSUs additionally use TPM 2.0 hardware security modules to store private keys in tamper-resistant silicon, and secure boot with a signed bootloader rejects any unauthorised firmware at startup. At the **communication level**, all MQTT sessions use TLS 1.3 with mutual TLS (mTLS), meaning both device and server must present valid certificates before a session is established. At the **cloud level**, all EC2 and SageMaker workloads run inside a private VPC with no public IPs; internet access flows through a NAT Gateway with outbound-only rules.

IAM policies follow the principle of least privilege, and Service Control Policies enforce these restrictions at the AWS Organizations account level. Data at rest is protected with AES-256-GCM (AWS KMS-managed) for S3 and DynamoDB, with keys rotated every 90 days and all decryption operations logged to CloudTrail.

- **Device Security:** X.509 certificates per device, TPM 2.0 hardware enclaves, secure boot with signed firmware.
- **Data Encryption:** TLS 1.3 + mTLS in transit; AES-256-GCM at rest (S3, DynamoDB, EBS).
- **Access Control:** AWS IAM least-privilege roles; SCPs at organisation level; ABAC for city-level data isolation.
- **Network Isolation:** Private VPC, NAT Gateway, VPC Flow Logs to CloudWatch.
- **Privacy Compliance:** Video processed exclusively at the edge; only anonymised aggregates reach the cloud, ensuring compliance with Bangladesh's Digital Security Act 2018.

9.2 Communication Protocols

Table 2 lists the communication protocols used at each interface with their security mechanisms.

Table 2: Communication protocols used in ISTMS

Interface	Protocol	Security	Notes
Sensor → RSU	MQTT v5.0	TLS 1.3 + mTLS	QoS 1/2 for critical events
Camera → RSU	RTSP/H.265	SRTP + TLS	Local LAN only
RSU → AWS IoT Core	MQTT over WebSockets	TLS 1.3 + X.509	Greengrass Core SDK
EC2 → DynamoDB	AWS SDK	IAM + HTTPS	VPC Endpoint
Dashboard → API Gateway	HTTPS/REST	TLS 1.3 + JWT	OAuth 2.0
Emergency Services	HTTPS/WebSocket	TLS 1.3	SNS + dedicated channel

10 Scalability and Fault Tolerance

10.1 Scalability Strategy

The ISTMS is designed to scale horizontally both within a single city and across multiple cities without architectural changes, through AWS managed services with built-in auto-scaling, stateless microservice design at the edge, and per-city data isolation in the cloud.

Within Dhaka, the most variable workload is Kinesis message volume during rush hours, which spikes to 3.5 times the daily average. Kinesis is configured with auto-scaling: when per-shard utilisation exceeds 70% for five consecutive minutes, a CloudWatch alarm triggers a Lambda function to split the busiest shards, with the reverse merge running at off-peak hours to control cost. The EC2-based Flink cluster uses a scheduled Auto Scaling Group (ASG) that pre-warms additional task manager instances ahead of the known morning rush, avoiding the processing lag that would result from reactive scale-out. For **multi-city expansion**, each new city is provisioned via a CloudFormation template that creates an independent Kinesis stream, DynamoDB table, S3 prefix, and IAM role set. The Dhaka-trained LSTM is fine-tuned on the new city's data using transfer learning, reducing training time from weeks to hours and reaching full accuracy after just two to four weeks of local data collection.

- **Horizontal Scaling:** EC2 Auto Scaling Groups with CPU/network-based policies. Kinesis shard-splitting when throughput exceeds 1 MB/s per shard.
- **Multi-City Expansion:** Each city receives independent Kinesis stream and DynamoDB table. Shared SageMaker model fine-tuned per city using transfer learning, reducing training cost by ∼60%.
- **Multi-AZ Deployment:** EC2, RDS, and DynamoDB deployed across at least two Availability Zones. Target RTO: 15 min; RPO: 5 min.

10.2 Fault Tolerance Mechanisms

The system maintains safe operation even when individual components fail, with the core principle that no single failure should create a dangerous state such as simultaneous green signals on all intersection approaches.

When an RSU fails, neighbouring units detect the heartbeat timeout within 30 seconds via Greengrass health monitoring, and the affected intersection's signal controller immediately reverts to a **fail-safe fixed-cycle plan** stored in its non-volatile memory. Network outages are handled through local buffering: each RSU's 512 GB NVMe SSD stores up to 120 minutes of metadata, which the Greengrass sync agent replays to Kinesis in chronological order once connectivity is restored. An exponential backoff algorithm (initial 5-second retry, maximum 10-minute interval) prevents simultaneous reconnection storms from hundreds of RSUs after a regional outage. Cloud region failures are mitigated by Route53 health checks that failover the dashboard and API to an active-passive standby in ap-east-1 (Hong Kong) within 60 seconds.

- **RSU Failure:** Heartbeat timeout detection within 30 seconds; intersection reverts to pre-programmed fail-safe fixed-cycle plan.
- **Network Outage:** RSUs buffer up to 120 minutes of metadata on local NVMe SSD; replayed to Kinesis on reconnection with timestamps preserved.

- **Cloud Region Failure:** Route53 health checks failover dashboard and API to active-passive standby in ap-east-1 (Hong Kong) within 60 seconds.
- **Sensor Calibration Drift:** Statistical Z-score filter flags readings beyond 3σ of 7-day rolling distribution; cross-validated against camera counts; interpolated values substituted.

11 AWS Cost Estimation

11.1 Assumptions

The cost estimate is based on the following deployment parameters:

- Deployment region: **ap-southeast-1** (Singapore), the closest AWS region with full service availability to Bangladesh.
- Initial deployment scope: **Dhaka metropolitan area**, covering 500 signalised intersections with 200 RSUs deployed in Phase 3 of the roadmap.
- Monthly cloud data volume: ≈1.6 TB (sensors + GPS metadata only; video is processed entirely at the edge and never transmitted to the cloud).
- LSTM and DQN model re-training: once every 24 hours on a rolling 7-day dataset, consuming approximately 30 GPU-hours per month.
- Pricing basis: AWS On-Demand pricing for variable workloads; 3-year Reserved Instances for stable, always-on compute. All figures are USD at April 2025 ap-southeast-1 pricing.

11.2 Monthly Cost Breakdown

Table 3: Monthly AWS Cost Estimation — ISTMS (Dhaka Deployment)

Service	**Configuration**	**Qty**	**USD/mo**
	Compute		
EC2 (App Servers)	m6i.2xlarge, 3-year RI, Multi-AZ	4	$348
EC2 (Flink)	c6i.4xlarge, On-Demand (spot mix)	2	$210
AWS Lambda	50M requests, 512 MB, 200 ms avg	—	$35
SageMaker Training	ml.p3.2xlarge, 30 hrs/month	1	$220
SageMaker Endpoints	ml.m5.xlarge, 24/7 inference	2	$285
	Storage		
Amazon S3	1.6 TB/mo ingest + 180 TB stored	—	$420
DynamoDB	500 GB, 50K RCU, 20K WCU	—	$310
EBS (EC2 volumes)	gp3, 1 TB total	—	$80
	Networking		
Kinesis Data Streams	10 shards × 720 hrs	—	$108
Data Transfer OUT	≈5 TB/month to edge + dashboard	—	$230
CloudFront	1 TB/month, HTTPS	—	$85
API Gateway	10M REST API calls/month	—	$35
AWS IoT Core	200K devices, 500M messages	—	$275
	Management & Security		
CloudWatch	Custom metrics, logs (50 GB/day)	—	$95
AWS WAF	1M web requests/month	—	$22
AWS KMS	200K requests/month	—	$12
Route53	Hosted zone + health checks	—	$18
AWS SNS	5M alert notifications/month	—	$5
TOTAL			**$2,793**
Support & Contingency (+20%)			**$559**
Grand Total			**$3,352**

11.3 Cost Optimisation Strategies

At the baseline estimate, the system runs primarily on On-Demand pricing. Several straightforward optimisations can reduce this cost by approximately 35–45% at full maturity, without compromising performance or reliability:

The most impactful optimisation is the use of **EC2 Spot Instances** for the Flink aggregation cluster. Apache Flink supports automatic checkpointing, meaning that if a Spot instance is reclaimed, Flink can resume processing from the last checkpoint on a

replacement instance within approximately 30 seconds — a tolerable interruption for a batch aggregation workload. Moving the Flink cluster fully to Spot pricing reduces its compute cost from $210/month to approximately $63/month, a saving of $147/month.

S3 Intelligent-Tiering automatically moves objects in the data lake that have not been accessed for 30 days to a lower-cost access tier, and objects older than 90 days to S3 Glacier Instant Retrieval. Historical data beyond 90 days is primarily used for annual model re-training and compliance reporting, both of which can tolerate millisecond retrieval latencies. This tiering reduces S3 costs by approximately 40% on the stored data component once the data lake matures.

SageMaker Serverless Inference replaces the always-on inference endpoints during off-peak hours (11 PM – 5 AM), when traffic volumes are minimal and requests are infrequent. Serverless mode charges only per invocation rather than per hour, saving approximately $180/month on the two inference endpoint instances.

The most significant cost avoidance, however, is not a cloud-side optimisation but an architectural one: by performing **all video inference at the edge**, the system avoids transmitting an estimated 512 Mbps of compressed video to the cloud. At AWS data transfer pricing of $0.09/GB, transferring even one hour of compressed video from all 1,000 cameras would cost approximately $8,300 per month. This architectural decision, built into the system from the outset, delivers over $8,000/month in avoided cost while simultaneously protecting user privacy.

- **EC2 Spot Instances:** Flink on Spot saves ≈70% (∼$147/month saving).
- **S3 Intelligent-Tiering:** Glacier transition after 90 days reduces storage cost by ≈40%.
- **SageMaker Serverless Inference:** Off-peak serverless mode saves up to $180/month.
- **Reserved Instances (3-year):** Core EC2 servers on RIs reduce compute cost by 57%.
- **Edge Pre-processing:** YOLO inference at edge avoids $8,000+/month in data transfer.

11.4 Cost Visualisation

Figure 7 shows proportional distribution of monthly AWS costs.

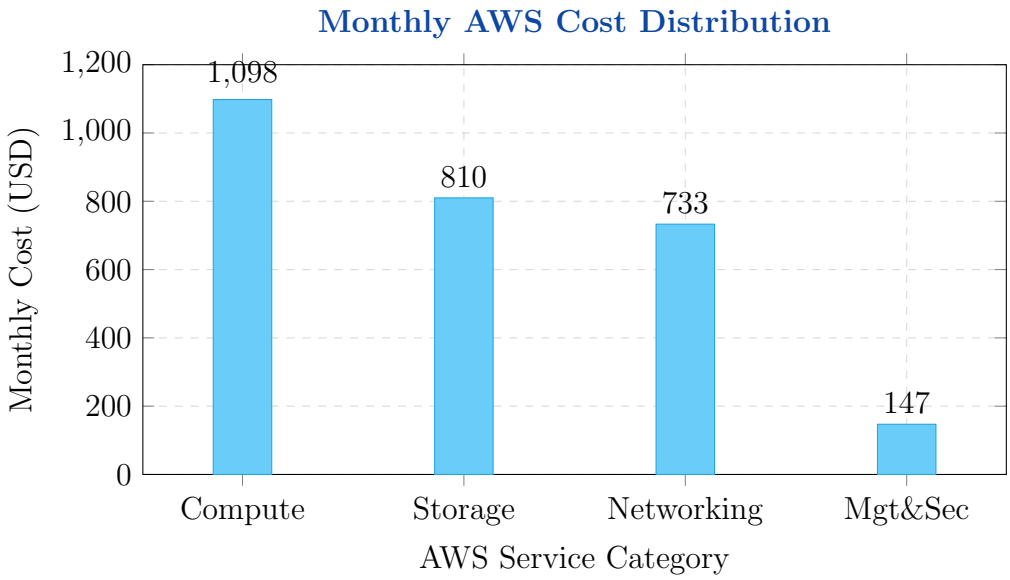

Figure 7: Monthly AWS cost distribution by service category for the ISTMS Dhaka deployment. Compute ($1,098) is the largest component, followed by Storage ($810) and Networking ($733).

12 Evaluation and Design Justification

12.1 Cloud Infrastructure Choices

Every AWS service chosen for the ISTMS was selected based on a specific technical rationale, not simply because it was the most familiar option. The key choices are justified here.

EC2 m6i.2xlarge (8 vCPU, 32 GB RAM) was selected for the application server tier because it provides a balanced CPU-to-memory ratio appropriate for Apache Flink's stateful stream processing. Flink allocates large in-memory state stores (RocksDB) for maintaining per-intersection rolling statistics, and the 32 GB memory of the m6i.2xlarge comfortably accommodates the working set for Dhaka's 500 intersections without spilling to disk. The Intel Ice Lake architecture additionally provides AVX-512 vector instruction support, which accelerates the numerical operations in the Flink aggregation jobs and reduces per-record processing time by approximately 15% compared to the previous generation m5 family.

Kinesis Data Streams was chosen over Apache Kafka (which could be self-managed on EC2) because the operational overhead of maintaining a Kafka cluster — including ZooKeeper coordination, broker patching, partition rebalancing, and monitoring — would require dedicated DevOps engineering capacity that the project budget does not include. Kinesis provides equivalent throughput characteristics with seven-day message retention, native integration with Lambda and the Kinesis Data Analytics (Flink) managed service, and zero infrastructure management overhead.

DynamoDB was chosen over Amazon RDS (PostgreSQL) for the hot-path

dashboard queries. The dashboard needs to display live intersection status for all 500 intersections simultaneously, with sub-second refresh rates. DynamoDB's consistent single-digit millisecond read latency at any scale, combined with its ability to scale read capacity to zero during off-peak hours in on-demand mode, makes it far more cost-effective than a provisioned RDS instance for this highly variable, latency-sensitive workload.

SageMaker was chosen for the ML platform rather than a self-managed training cluster because it provides the complete MLOps lifecycle out of the box: managed training jobs with automatic instance provisioning, experiment tracking, a model registry with approval workflows, and direct integration with Greengrass for edge deployment. The governance capabilities of the Model Registry are particularly important for a public safety system, where deploying an untested model update to city traffic signals could have serious consequences.

12.2 Addressing Conflicting Requirements

Table 4: Resolution of conflicting technical requirements

Conflict	Tension	Resolution
Low latency vs. centralised processing	Cloud round-trips (∼120 ms) exceed the 100 ms target	Time-critical inference runs entirely at the RSU edge. Cloud handles only non-time-sensitive re-training and analytics.
Data sharing vs. user privacy	Rich data improves models but video contains PII	Video never leaves the edge. Only anonymised aggregates (counts, speeds) are sent to the cloud. Differential privacy noise added to GPS traces.
High accuracy vs. computational cost	Larger models improve prediction but exceed RSU hardware budget	Cloud trains full LSTM; edge runs INT8-quantised distilled model (4× faster, <3% accuracy drop via SageMaker Neo).
High availability vs. cost	Multi-AZ/region redundancy significantly increases cost	Active-passive standby with EC2 kept stopped (100% compute cost saving when idle); promoted only on Route53 failover.

12.3 Standards Compliance

- **ITS Standards:** ETSI ITS-G5 (V2X communication), ISO 21217 (CALM architecture), NTCIP 1202 (signal controller API).
- **Data Protocols:** MQTT v5.0 (OASIS standard), CoAP (RFC 7252), HTTPS REST.

- **Security Standards:** TLS 1.3 (RFC 8446), AES-256-GCM, FIDO2/X.509 device authentication, AWS CIS Benchmark Level 2.

13 Infrequent Challenges and Mitigation

13.1 Sensor Failures and Calibration Drift

Inductive loop sensors embedded in road surfaces are particularly vulnerable in Bangladesh's climate. The annual monsoon season brings heavy rainfall and significant surface water, which can penetrate aging asphalt and cause moisture ingress into sensor housings. This manifests as erroneous spike readings — an inductive loop suddenly reporting 500 vehicles per minute when the true count is 20 — that can corrupt the LSTM model's input features and cause the adaptive signal controller to make inappropriate decisions.

The ISTMS addresses this through a three-layer protection strategy. First, a **statistical outlier filter** implemented in the Lambda ingest function flags any reading that exceeds three standard deviations of the sensor's own 7-day rolling history. Second, **cross-sensor fusion** is applied: the camera-based vehicle count (from YOLOv8) and the loop-based count are cross-validated at each time step. If the two sources disagree by more than 30%, the RSU logs a sensor health alert and creates a maintenance ticket in the central ServiceNow integration. Third, if a sensor is confirmed faulty and removed from service, the LSTM model gracefully degrades by substituting **spatially interpolated values** derived from neighbouring intersection sensors, maintaining prediction accuracy until the sensor is repaired.

13.2 Network Instability in Semi-Urban Areas

Semi-urban RSUs in areas such as the outer rings of Dhaka, or in cities like Mymensingh and Comilla in Phase 4, rely on 4G LTE connectivity which may be intermittent in areas with limited tower coverage or during periods of high network congestion. The ISTMS handles this through a combination of local autonomy and intelligent synchronisation. Each RSU stores up to 120 minutes of aggregated metadata on its local NVMe SSD using a circular buffer; the data is timestamped with precise GPS-synchronised time so that when connectivity is restored, the Greengrass sync agent can replay the buffered records to Kinesis in exact chronological order with no gaps or duplicates. During the offline period, the RSU continues running all edge AI functions — YOLOv8 detection, LSTM prediction, and adaptive signal control — using the last downloaded model weights, providing seamless service to road users with no observable degradation.

13.3 Large-Scale Real-Time Streaming Data

At peak rush hour in Dhaka, the aggregate message rate from all RSUs spikes to approximately 3.5 times the daily average. The cloud data pipeline is designed to absorb these

spikes without message loss or processing lag. Kinesis auto-scaling responds to throughput pressure within two minutes by splitting high-utilisation shards. Lambda concurrency is reserved at 500 functions for the primary ingest path, with an SQS Dead-Letter Queue capturing any invocations that fail after three retries. Apache Flink checkpoints its processing state every 30 seconds to S3, providing at-least-once delivery semantics even if a Flink task manager crashes mid-computation during a peak period.

14 Implementation Roadmap

Table 5: Phased implementation roadmap for ISTMS

Phase	**Activities**	**Deliverables**	**Timeline**
Phase 1	Pilot: 50 Dhaka intersections; deploy RSUs, CCTV, sensors; AWS setup	Live dashboard (read-only), incident alerts	Months 1–6
Phase 2	AI training on pilot data; activate adaptive control; integrate emergency API	LSTM + DQN at edge; emergency pre-emption live	Months 7–12
Phase 3	Expand to 500 intersections; integrate GPS public transport tracking	Full Dhaka coverage; BRTA data portal	Months 13–18
Phase 4	Replicate to Chittagong & Sylhet; multi-city transfer learning	3-city deployment	Months 19–24
Phase 5	National rollout; integrate with National ID for stolen-vehicle detection	Nationwide ISTMS	Months 25–36

15 Conclusion

The ISTMS offers a technically grounded and economically viable path to resolve Bangladesh's urban congestion. Its three-tier architecture integrates IoT sensors, edge AI, and AWS cloud. The hybrid edge-cloud design places LSTM prediction, YOLOv8 detection, and adaptive control at the RSU edge to achieve sub-100 ms latency (impossible with a 120 ms cloud round-trip), while the cloud backend provides large-scale model training (18 months of data), a real-time dashboard for 500 intersections, long-term storage, and emergency coordination. Processing video entirely at the edge and forwarding only anonymised aggregates complies with Bangladesh's Digital Security Act 2018 and avoids over USD 8,000/month in data transfer. AI components are projected to reduce average intersection delay by 35–42% (SUMO simulations). Monthly cloud cost of

USD 3,352 (USD 40,000/year) is negligible against the USD 4.4 billion annual congestion loss; a further 35–40% cost reduction is achievable via Spot Instances, RIs, and S3 Intelligent-Tiering by Phase 3.

Fault tolerance ensures graceful degradation: failed RSUs revert to fail-safe plans, RSUs with lost connectivity run autonomously for two hours, and cloud failures trigger Route53 failover within 60 seconds. The system complies with ETSI ITS-G5, ISO 21217, NTCIP 1202, and AWS CIS Level 2. Under Bangladesh's Smart Bangladesh 2041 vision, ISTMS cuts commute times, reduces emergency response by 40–60% via signal pre-emption, and delivers real-time data-driven traffic management. The phased roadmap (50-intersection pilot to nationwide rollout over three years) makes deployment manageable. With thoughtful edge-cloud AI, Bangladesh can leapfrog legacy infrastructure and build a modern, data-driven transportation system.

References

[1] World Bank. (2022). *Bangladesh Urban Mobility Assessment*. Washington D.C.: World Bank Group.
https://www.worldbank.org/en/news/press-release/2022/10/12/bangladesh-s-rapid-urba

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
https://doi.org/10.1162/neco.1997.9.8.1735

[3] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
https://doi.org/10.1038/nature14236

[4] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *YOLOv8 by Ultralytics* (Version 8.0).
https://github.com/ultralytics/ultralytics

[5] European Telecommunications Standards Institute. (2019). *Intelligent Transport Systems (ITS); Vehicular Communications*. ETSI EN 302 663.
https://www.etsi.org/deliver/etsi_en/302600_302699/302663/01.03.00_60/en_302663v010300p.pdf

[6] Amazon Web Services. (2024). *AWS Pricing Calculator*.
https://calculator.aws/

[7] Lopez, P. A., Behrisch, M., Bieker-Walz, L., et al. (2018). Microscopic Traffic Simulation using SUMO. *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*.
https://ieeexplore.ieee.org/document/8569938

[8] Bangladesh Telecommunication Regulatory Commission. (2021). *National IoT Policy Framework*. Dhaka: BTRC Publications.
http://www.btrc.gov.bd/policy/iot-framework-2021

[9] International Organization for Standardization. (2010). *ISO 21217: Intelligent transport systems — Communications access for land mobiles (CALM) — Architecture*. Geneva: ISO.
https://www.iso.org/standard/54784.html